КОМП’ЮТЕРНИЙ ПРАКТИКУМ №4

Тема: Вивчення криптосистеми RSA та алгоритму електронного підпису; ознайомлення з методами генерації параметрів для асиметричних криптосистем

Мета роботи: Ознайомлення з тестами перевірки чисел на простоту і методами генерації ключів для асиметричної криптосистеми типу RSA; практичне ознайомлення з системою захисту інформації на основі криптосхеми RSA, організація з використанням цієї системи засекреченого зв'язку й електронного підпису, вивчення протоколу розсилання ключів.

Мельниченко Артемій ФБ-41

---

1. **Написати функцію пошуку випадкового простого числа з заданого інтервалу або заданої довжини, використовуючи датчик випадкових чисел та тести перевірки на простоту. В якості датчика випадкових чисел використовуйте вбудований генератор псевдовипадкових чисел вашої мови програмування. В якості тесту перевірки на простоту рекомендовано використовувати тест Міллера-Рабіна із попередніми пробними діленнями. Тести необхідно реалізовувати власноруч, використання готових реалізацій тестів не дозволяється.**

```
Пошук простого числа
Кількість перевірених кандидатів: 13
Кількість відхилених кандидатів: 12

Перші п'ять відхилених кандидатів:
109349038898869132208183533501877366009037114311651313531802644391945738027805 —
 ділиться на 5
989465829375702094661831370532009155975764809097832112144508247368925479223 67 —
не пройшло тест Міллера-Рабіна
613120637533594292012017905789376237095265348040793190895438629732325396057 9 —
не пройшло тест Міллера-Рабіна
901355059507521131640575316896358057613866904726168531650215796991659842221 19 —
не пройшло тест Міллера-Рабіна
103938310056885180509378258685391043204446629829092408024645145806008085902993 —
 ділиться на 37

Знайдене число p = 802280749040635133253849295989853918023466153620530761503244411026608935202 63
Довжина: 256 біт
Кількість раундів тесту Міллера-Рабіна: 32

Проміжні результати першого раунду:
p - 1 = 2^1 * 401140374520317566626924647994926959011733076810265380751622205513304467601 31
s = 1
d = 401140374520317566626924647994926959011733076810265380751622205513304467601 31
Основа a = 750133970720807596520908961061536355248849535219453077720115415593645128592 2
x_0 = a^d mod p = 1
x_0 = 1 — раунд пройдено.

Усі 32 раунди пройдено: число є ймовірно простим.
Верхня межа ймовірності проходження тесту складеним числом:
4^(-32) = 5.421011e-20
```

У програмі task1.py реалізовано пошук випадкового простого числа довжиною 256 біт. Кандидати генеруються за допомогою модуля random. Старший біт встановлюється в 1 для забезпечення заданої довжини, молодший - для отримання непарного числа. Спочатку виконуються пробні ділення на малі прості числа від 2 до 37, після чого 32 раунди власної реалізації тесту Міллера-Рабіна з випадковими основами. Під час запуску перевірено 13 кандидатів, із яких 12 відхилено.
Для знайденого числа p подано розклад p - 1 = 2^s * d, де s = 1, а d - непарне. Для наведеної у виводі основи a отримано a^d mod p = 1, тому цей раунд пройдено без додаткових піднесень до квадрата.
Знайдене число пройшло всі 32 раунди та вважається ймовірно простим. Імовірність того, що фіксоване складене число пройде всі раунди за незалежного випадкового вибору основ, не перевищує 4^(-32), що приблизно дорівнює 5,42 * 10^(-20).

---

2. **За допомогою цієї функції згенерувати дві пари простих чисел p, q і p_1, q_1 довжини щонайменше 256 біт. При цьому пари чисел беруться так, щоб pq ≤ p_1q_1; p і q - прості числа для побудови ключів абонента A, p_1 і q_1 - абонента B**

```
Прості числа абонента A:
p = 98704659918037387990223850306180900251483061275578520222453929634493031510667
Довжина p: 256 біт
q = 63276115078251912642089400957396051832650304889308091494421654270675346239141
Довжина q: 256 біт

Прості числа абонента B:
p1 = 10180808642168212156888910413201209934480436249284799828306465059873800300631
7
Довжина p1: 256 біт
q1 = 79931323095162748211888588165274727689459949978635022826918915278695168359651
Довжина q1: 256 біт

p * q = 62456474197334527620424280839136687848002939841153338673819464708907972700
8153182969456002057705427000873431415923029489161327506900680867899096427441704 7
p1 * q1 = 81376550494717251565288709002205532931028467110349759159997234950709755
0530030883522744371489016712466966830736353344499632303320346477437174289548091536
7
Умова p * q <= p1 * q1: True

Числа збережено у primes.json
```

У програмі task2.py за допомогою функції generate_prime() з першого завдання згенеровано чотири різні ймовірно прості числа p, q, p1 та q1. Кожне число має довжину 256 біт і пройшло пробні ділення та 32 раунди тесту Міллера-Рабіна.
Числа розподілено між абонентами A і B так, щоб виконувалася умова p * q <= p1 * q1. Якщо добуток першої пари більший, програма міняє пари місцями. За результатами запуску умова виконується, що підтверджується значенням True у виводі. Отримані числа збережено у файлі primes.json для подальшої генерації ключів RSA.

---

3. Написати функцію генерації ключових пар для RSA. Після генерування функція повинна повертати та/або зберігати секретний ключ (d, p, q) та відкритий ключ (n, e). За допомогою цієї функції побудувати схеми RSA для абонентів A і B - тобто, створити та зберегти для подальшого використання відкриті ключі (e, n), (e_1, n_1) та секретні d і d_1.

```
Абонент A:
p = 98704659918037387990223850306180900251483061275578520222453929634493031510667
q = 63276115078251912642089400957396051832650304889308091494421654270675346239141
n = 62456474197334527620424280839136687848002939841153338673819464708907972700815
31829694560020577054270008734314159230294891613275069006808678990964274417047
Довжина n: 511 біт
phi(n) = 62456474197334527620424280839136687848002939841153338673819464708907972700
81369848919563731276421956757470737207146161525448388457289933095085795896667240
e = 65537
d = 54297878661936494586283989579789308738999580395647536876566486431401203176855
23116896425883900779977847836378276612595924042104166235122572372984833124993

Перевірка співвідношень RSA:
p != q: True
n = p * q: True
1 < e < phi(n): True
НСД(e, phi(n)) = 1
(e * d) mod phi(n) = 1

Абонент B:
p = 10180808642168212156888910413201209934480436249284799828306465059873800300631
7
q = 79931323095162748211888588165274727689459949978635022826918915278695168359651
n = 81376550494717251565288709002205532931028467110349759159997234950709755053003
08835227443714890167124669668307363533444996323033203464774371742895480915367
Довжина n: 512 біт
phi(n) = 81376550494717251565288709002205532931028467110349759159997234950709755053
00127095817926870020386346977371020536499180683851550182354790805865462309549400
e = 65537
d = 28219858447799547985478683636619392815104200253550192661080872769958658514878
00608917924591832908441151635124948243234804795675435164522798494047056000873

Перевірка співвідношень RSA:
p != q: True
n = p * q: True
1 < e < phi(n): True
НСД(e, phi(n)) = 1
(e * d) mod phi(n) = 1

Ключі абонентів A і B збережено у keys.json
```

У програмі task3.py реалізовано функцію GenerateKeyPair(p, q), яка повертає відкритий ключ (n, e) та секретний ключ (d, p, q). Для обчислень використано прості числа з файлу primes.json. Модуль визначається як n = p * q, функція Ойлера - phi(n) = (p - 1) * (q - 1). Для обох абонентів обрано e = 65537, а секретний показник d обчислено як обернений до e за модулем phi(n). Для абонента A отримано модуль довжиною 511 біт, для абонента B - 512 біт. Для обох пар підтверджено, що p != q, n = p * q, 1 < e < phi(n), НСД(e, phi(n)) = 1 та (e * d) mod phi(n)=1. Отже, необхідні співвідношення для сформованих ключів виконуються. Відкриті та секретні ключі збережено у файлі keys.json для подальшого шифрування, створення підписів і передавання ключа.

---

4. Написати програму шифрування, розшифрування і створення повідомлення з цифровим підписом для абонентів A і B. Кожна з операцій (шифрування, розшифрування, створення цифрового підпису, перевірка цифрового підпису) повинна бути реалізована окремою процедурою, на вхід до якої повинні подаватись лише ті ключові дані, які необхідні для її виконання.

```
Відкрите повідомлення M = 450830293991997939651915868797310228634644940959200855
176167986459747055964166416456516230219145034541528411587019827329069201783579886
312362468284986337

Абонент A:
Шифротекст C = 448532951889934586452889811497336534925929314919947310046217387503
775652000803250403275804390349902097654030432338394713418508505752828560238754574
6806513
Розшифроване повідомлення M' = 450830293991997939651915868797310228634644940959200
855717616798645974705596416641645651623021914503454152841158701982732906920178357
79886312362468284986337
M' = M: True
Цифровий підпис S = 129639347873744710470188968070071126251728289521345642760548
640139164427133299198258945112352950324165809576765205078494947201777237428135480
224040326238
Підписане повідомлення (M, S) = (450830293991997939651915868797310228634644940959200
855717616798645974705596416641645651623021914503454152841158701982732906920178357
98863123624682849863337, 129639347873744710470188968070071126251728289521345642760
548864013916442713329919825894511235295032416580957676520507849494720177723742813
540224040326238)
Результат перевірки підпису: True

Абонент B:
Шифротекст C = 223379602265640526864793594306003902045238076610267656393499310923
468351631906588489086659293558203820878520247699361276762343479279216456405854463
7217848
Розшифроване повідомлення M' = 450830293991997939651915868797310228634644940959200
855717616798645974705596416641645651623021914503454152841158701982732906920178357
79886312362468284986337
M' = M: True
Цифровий підпис S = 410452191458748188523018438781920468994191681909825685390909
509539379377418019690624398494657958934320707420043231961120990298758225466449994
58017814382
Підписане повідомлення (M, S) = (450830293991997939651915868797310228634644940959200
855717616798645974705596416641645651623021914503454152841158701982732906920178357
98863123624682849863337, 410452191458748188523018438781920468994191681909825685389
809190953937937741801969062439849465795893432070742004323196112099029875822546644
99945801781438 2)
Результат перевірки підпису: True
```

У програмі task4.py реалізовано окремі функції Encrypt(), Decrypt(), Sign() та Verify(). Для шифрування і перевірки підпису використовуються лише відкриті параметри n та e, для розшифрування і створення підпису - n та секретний показник d. Ключі абонентів завантажуються з файлу keys.json. Повідомлення M обрано випадково в межах 1 <= M < min(n, n1), щоб його можна було використати для обох абонентів.
Шифрування виконується за формулою C = M^e mod n, розшифрування - M’ = C^d mod n. Цифровий підпис обчислюється як S = M^d mod n, після чого формується підписане повідомлення (M, S). Для перевірки підпису перевіряється рівність S^e mod n = M. Для абонентів A і B розшифровані повідомлення збіглися з початковим, а перевірка обох підписів повернула True. Це підтверджує правильність виконання операцій для обраного повідомлення.

---

5. За допомогою раніше написаних на попередніх етапах програм організувати роботу протоколу конфіденційного розсилання ключів з підтвердженням справжності по відкритому каналу за допомогою алгоритму RSA.

```
Передавання ключа від абонента A до абонента B
Умова n <= n1: True

Абонент A:
Випадковий ключ k = 3025986118526178744334133643999846938491858009090022754772062
810613098220855872631415405901554504384909182884540928088106850694534172055459965
009558293623
Підпис S = k^d mod n = 2077029206586717549216807362239676707340368416852776923191
607069663480626978152804423848449534934848517286861715156830650353753213351733902
486813722176971

Передані відкритим каналом значення:
k1 = k^e1 mod n1 = 9008776255407353292860243420640653106498753321235317157657004 6
071183350172618454809849683798386267807888826519023809064258854688913950166305492
9543633056
S1 = S^e1 mod n1 = 7596940478325147059250021417887433101283395299036423551486332 0
788194714525833737806941919074288893576006880375472644218254517992628723726701447
82978708373

Абонент B:
Відновлений ключ k = k1^d1 mod n1 = 3025986118526178744334133643999846938491858 00
909002275477206281061309822085587263141540590155450438490918288454092808810685069
453417205545996500955829 3623
Відновлений підпис S = S1^d1 mod n1 = 2077029206586717549216807362239676707340368
416852776923191607069663480626978152804423848449534934848517286861715156830650353
753213351733902486813722176971
S^e mod n = 3025986118526178744334133643999846938491858009090022754772062810613 09
822085587263141540590155450438490918288454092808810685069453417205545996500955829
3623
Підпис підтверджено: True
Отриманий ключ збігається з початковим: True

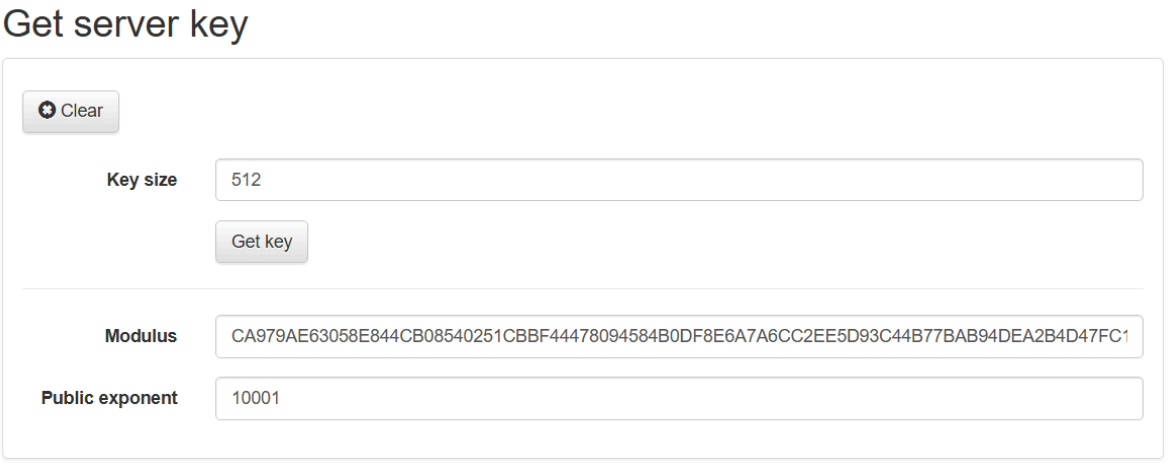
Конфіденційність:
Ключ і підпис зашифровано відкритим ключем B (n1, e1).
Для розшифрування B використовує свій секретний показник d1.

Підтвердження справжності відправника:
A створює підпис своїм секретним показником d.
B перевіряє підпис відкритим ключем A (n, e).
```

У програмі task5.py реалізовано функції SendKey() та ReceiveKey(), які отримують лише необхідні для роботи ключові дані. Для передавання від A до B випадково обрано ключ k у межах 0 < k < n. Умова n <= n1 виконується. Абонент A створює підпис S = k^d mod n, після чого шифрує ключ і підпис відкритим ключем B: k1 = k^e1 mod n1, S1 = S^e1 mod n1. Відкритим каналом передається пара (k1, S1). Абонент B відновлює ключ і підпис за допомогою свого секретного показника d1 та перевіряє рівність S^e mod n = k відкритим ключем A. Перед прийняттям ключа перевіряються допустимі межі отриманих значень і правильність підпису. За результатами запуску підпис підтверджено, а отриманий ключ збігається з початковим. Конфіденційність забезпечується шифруванням відкритим ключем B і розшифруванням його секретним ключем. Справжність відправника підтверджується підписом, створеним секретним ключем A та перевіреним його відкритим ключем.

Для перевірки сумісності власної реалізації RSA з тестовим середовищем виконано шифрування та розшифрування в обох напрямках. На сервері отримано відкритий ключ із модулем довжиною 512 біт. Значення відкритого показника 10001 у

шістнадцятковій системі відповідає числу 65537 у десятковій.

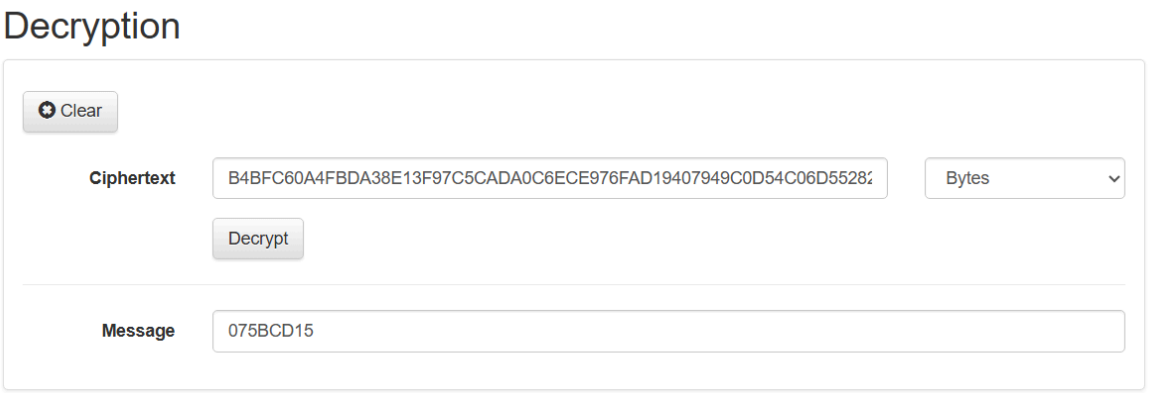


Модуль сервера n у HEX:
CA979AE63058E844CB08540251CBBF44478094584B0DF8E6A7A6CC2EE5D93C44B77B
AB94DEA2B4D47FC1F4C5584A6ABE65E3BA171A58F347443B44B64B168E45
Отримання відкритого ключа тестового сервера.

У програмі server_test.py повідомлення M = 123456789 зашифровано відкритим ключем сервера за допомогою функції Encrypt(). Отриманий шифротекст передано серверу через форму Decryption у режимі Bytes. Сервер повернув 075BCD15 у шістнадцятковій системі, що відповідає початковому повідомленню 123456789. Початковий нуль не змінює числового значення.

```
Повідомлення DEC: 123456789
Повідомлення HEX: 75BCD15
Шифротекст HEX: B4BFC60A4FBDA38E13F97C5CADA0C6ECE976FAD19407949C0D54C06D55282D2E
E3C699871167309F2582831507DF829DACD3FE35DDA18A3BD3D893CD75B0F413
```

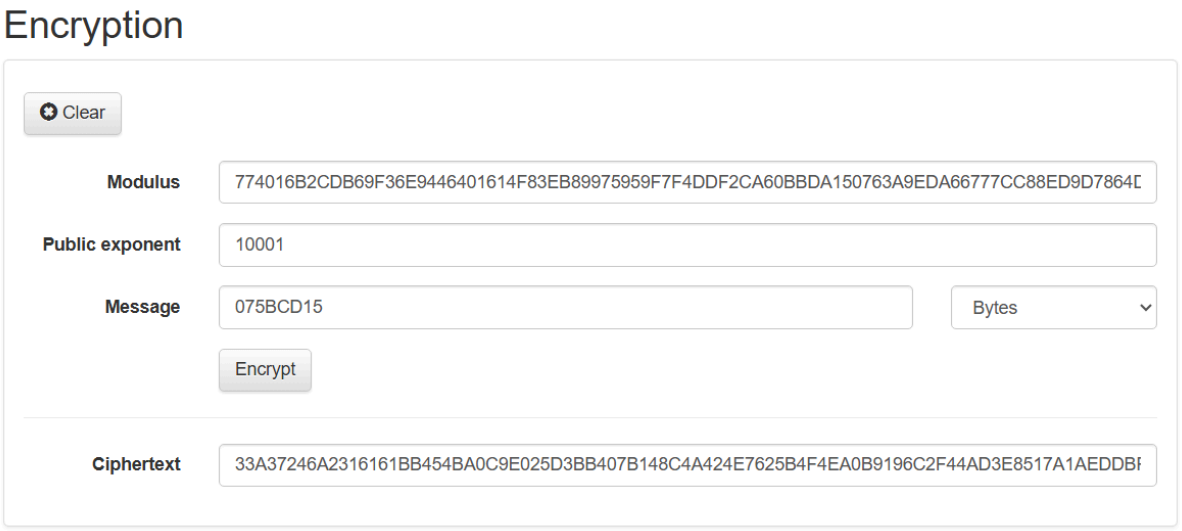
Шифрування повідомлення власною програмою для тестового сервера.



Успішне розшифрування повідомлення на тестовому сервері.

Для перевірки зворотного напрямку у форму Encryption введено відкритий ключ абонента A та повідомлення 075BCD15 у режимі Bytes. Отриманий від сервера шифротекст розшифровано програмою server_decrypt.py за допомогою функції

Decrypt() і секретного показника абонента A. Відновлено число 123456789, а перевірка збігу з початковим повідомленням повернула True.



Шифротекст сервера у HEX:
33A37246A2316161BB454BA0C9E025D3BB407B148C4A424E7625B4F4EA0B9196C2F44
AD3E8517A1AEDDBF748BE80079C2F058414667AA30D54E4818C1CDB405C

Шифрування повідомлення на сервері відкритим ключем абонента A.

```
Шифротекст сервера HEX: 33A37246A2316161BB454BA0C9E025D3BB407B148C4A424E7625B4F4
EA0B9196C2F44AD3E8517A1AEDDBF748BE80079C2F058414667AA30D54E4818C1CDB405C
Розшифроване повідомлення DEC: 123456789
Розшифроване повідомлення HEX: 75BCD15
Збіг із початковим повідомленням: True
```

Успішне розшифрування серверного шифротексту власною програмою.

Контрольні питання:

1. Під час генерації кандидатів старший біт встановлюється в 1 для забезпечення довжини 256 біт, а молодший - для отримання непарного числа. Далі виконуються пробні ділення на малі прості числа від 2 до 37, які дозволяють швидко відкинути частину складених кандидатів. Решта проходять 32 раунди тесту Міллера-Рабіна з випадковими основами. Якщо хоча б один раунд виявляє складеність, кандидат відхиляється. Число, яке пройшло всі раунди, вважається ймовірно простим.
2. Числа p і q повинні бути різними простими числами, n = p * q, phi(n) = (p - 1) * (q - 1). Відкритий показник e має задовольняти умови 1 < e < phi(n) та НСД(e, phi(n)) = 1. Секретний показник d повинен виконувати співвідношення (e * d) mod phi(n) = 1. У реалізації простота перевірялася під час генерації чисел, а їхня відмінність забезпечувалася відбором різних значень. Решту співвідношень перевірено в task3.py, результати виведено для обох абонентів.
3. Encrypt() отримує повідомлення M та відкриті параметри адресата n, e. Decrypt() отримує шифротекст C, модуль n і секретний показник d. Sign() отримує повідомлення M, модуль n і секретний показник d підписувача. Verify() отримує повідомлення M, підпис S та відкриті параметри підписувача n, e. Кожна

операція використовує лише параметри, які входять до її формули. Числа p, q та phi(n) для цих обчислень не потрібні.

4. Конфіденційність забезпечується шифруванням ключа k та його підпису S відкритим ключем абонента B. Для їх відновлення B використовує свій секретний показник d1. Справжність відправника підтверджується цифровим підписом: A створює його своїм секретним показником d, а B перевіряє відкритим ключем A рівність S^e mod n = k.
5. Одержувач перевіряє, що отримані шифротексти задовольняють умови 0 <= k1 < n1 та 0 <= S1 < n1. Після розшифрування перевіряються межі 0 < k < n і 0 <= S < n. Далі перевіряється підпис за рівністю S^e mod n = k. Якщо будь-яка перевірка не проходить, ключ не приймається. Порівняння отриманого ключа з початковим у головній програмі є додатковою перевіркою експерименту, оскільки під час реального обміну одержувач не знає початкового ключа.

Висновки:

У роботі реалізовано генерацію ймовірно простих чисел із використанням пробних ділень і тесту Міллера-Рабіна, створено ключові пари RSA для двох абонентів та перевірено необхідні співвідношення між параметрами. Виконано шифрування, розшифрування, створення та перевірку цифрових підписів. Для обох абонентів розшифроване повідомлення збіглося з початковим, а підписи успішно пройшли перевірку. Реалізовано протокол конфіденційного передавання ключа з підтвердженням справжності відправника. Під час локального експерименту одержувач правильно відновив переданий ключ і підтвердив підпис.

Додатково перевірено сумісність реалізації з тестовим середовищем: повідомлення, зашифроване власною програмою, успішно розшифровано сервером, а повідомлення, зашифроване сервером, правильно відновлено локально.

Під час виконання роботи суттєвих труднощів не виникло.